

System Product Requirements Document & Architecture Constraint Guide

System Name: Ghana Smart Service Operations Optimizer — Campus Dispatch & Optimization Platform

Target Context: University of Ghana (UG) Campus Taxi & Emergency Dispatch (UG Hospital Focal Node)

1. Product Description & Operational Overview

The system is a centralized campus mobility and emergency dispatch platform tailored for the University of Ghana context. It regulates and prioritizes campus taxi dispatches by identifying student congestion hotspots, verifying official campus drivers, and dynamically routing verified vehicles to high-density or high-urgency pickup points. The focal node of the transportation network is the **UG Hospital**, ensuring rapid transit links from residential hostels (e.g., Pentagon, Commonwealth, JSB, Limann) to medical care and academic hubs.

2. Priority Scoring Engine & Mathematical Formulation

To strictly enforce access rules, every service request R_i is evaluated by a dynamic Priority Calculator upon entry into the Custom Max-Heap Priority Queue. The priority score $P(R_i)$ determines queue ordering and preemptive dispatch privileges.

2.1 Priority Scoring Formula

$$P(R_i) = (\text{UserCategoryWeight} * W_{\text{cat}}) + (\text{MedicalUrgencyWeight} * W_{\text{urg}}) + (\text{WaitTimeMinutes} * W_{\text{wait}}) + (\text{DestinationBonus})$$

2.2 Weight Classifications & Rules

Category / Condition	Base Weight Score	Operational Behavior & Preemption Rules
Tier 1A: Ill / Emergency Students	1000 pts	Immediate Preemption: Directly overrides all standard queues. Dispatches nearest verified taxi immediately to transit to UG Hospital.
Tier 1B: Disabled Students	800 pts	High Priority Queue Jump: Placed ahead of all standard bookings regardless of submission time. Assisted vehicle matching.
Tier 2: Standard UG Students	400 pts	Base Student Priority: Ranked primarily by submission timestamp (FIFO within Tier 2) with incremental time-based escalation.
Tier 3: University Staff / Non-Students	100 pts	Lower Priority: Processed only when Tier 1 and Tier 2 queues are clear or when vehicle capacity in zone exceeds student demand.

2.3 Additional Priority Modifiers

- **Destination Bonus (+200 pts):** Automatically applied if the destination node is UG_Hospital, elevating routine medical visits above general campus transit.
- **Aging / Anti-Starvation Factor (+15 pts/min):** Added to WaitTimeMinutes every minute a request remains in queue to prevent Tier 3 requests from starving indefinitely.
- **Index Parameter Tie-Breaker:** If $P(R_a) == P(R_b)$, the tie-breaker is computed as $(RequestId \% TeamIndexParameter)$ to guarantee deterministic ordering.

3. System Architecture Requirements

3.1 Layered Architecture Diagram (Custom Java Stack)

Architecture Layer	Component Responsibilities	Underlying Custom Data Structures Used
Presentation / Console Layer	CLI Menu, Interactive Dispatch Controls, Real-Time Trace Logs, Administrative Metrics.	Custom String Buffers & Console Handlers.
Service Scheduling Engine	Request Ingestion, Preemption Logic, Priority Calculation, Cancellation Processing.	Custom Max-Heap (Priority Queue), Circular FIFO Queue, Custom Undo Stack.
Graph Routing Engine	Campus Road Network Modeling, Shortest Path Computation, Congestion Hotspot Detection.	Adjacency List (Node Linked Lists), Custom Disjoint Set, Dijkstra Engine.
Indexing & Lookup Layer	$O(1)$ Request Retrieval, Fast Driver Lookup, Active Booking Search.	Custom Hash Table (with Chaining Linked Lists), Binary Search Tree (BST).
Data Access Layer	DB Schema Management, CSV Seeding, Audit Event Logging, Benchmark Result Storage.	JDBC Bridge, Custom Dynamic Arrays (Row Mappers).

4. Implementation Constraints & Academic Rules

4.1 Strict Constraints (Syllabus Compliance)

- **Zero External Collection Frameworks:** Under no circumstances may `java.util.ArrayList`, `java.util.HashMap`, `java.util.PriorityQueue`, `java.util.LinkedList`, or `java.util.Stack` be used in core logic modules.
- **Database Persistence:** All initial data (50 locations, 100 roads, 300 requests, 30 resources) must seed from CSV into SQLite/MySQL/PostgreSQL, and runtime operations must persist back to the DB.
- **Index Number Derivation:** At least three system parameters (e.g., hash table capacity M , priority scale factor, road congestion penalties) must be explicitly derived from team member index numbers.
- **Trace & Proof Generation:** Execution traces must be logged to text/console showing front/rear queue pointers, heap array swaps during preemption, and Dijkstra node distance updates.

4.2 Non-Functional & Performance Targets

- **Search Latency:** Hash Table lookups for active ride requests must maintain $O(1)$ average time complexity under load factor $\alpha < 0.75$.
- **Shortest Path Execution:** Dijkstra pathfinding across 50+ nodes must execute in $O((V + E) \log V)$ using the custom min-heap.
- **Empirical Scaling Tests:** Benchmarking suite must run algorithms over inputs of $N = \{100, 500, 1000, 5000, 10000, 50000\}$ and export execution times (in nanoseconds) to CSV for plotting.

